Universidad de Chile
Facultad de Ciencias Físicas y Matemáticas
Departamento de Ingeniería Eléctrica

Proyecto Final

Clasificación ICU iPhone data set

Integrantes: Diego Vega
Profesor: Javier Ruiz del Solar
Auxiliar: Patricio Loncomilla
Ayudantes: Benjamín Llancao Mella
Bruno Cavieres L.
Gonzalo Olguín M.

Fecha de realización: 18 de julio de 2024
Fecha de entrega: 17 de julio de 2024
Santiago de Chile

Introducción

En el presente informe se detalla el desarrollo del proyecto final del curso Inteligencia Computacional. El desarrollo del proyecto se realizo acrode a las indicaciones del documento *Enunciado_proyecto_v2.pdf*.
En el documento mencionado se tiene por objetivo la implementación y análisis de, al menos, dos clasificadores distintos. El conjunto de datos utilizados durante el proyecto sera una modificación del *UCF-iPhone Dataset* de UCF Center for Research in Computer Vision, el cual fue entregado por el cuerpo docente en los archivos *df_smartphone_trainval.csv* y *df_smartphone_test.csv*. El conjunto de entrenamiento y validación posee 1125 caracteristicas, 8 clases y 4638 muestras en total.
Las secciones del proyecto son:

Gráficos de las señales Esta sección consiste en: En la generación de gráficos de las señales. En particular se realizaron gráficos de los sensores considerando los 3 ejes para cada clase y se realizaron gráficos de cada eje de los sensores para cada clase.

Eliminación de señales Esta sección consiste en: La argumentación de la no eliminación de señales.

Filtrado de señales Esta sección consiste en: La visualización de las señales con distintos tipos de filtros y la aplicación de un filtro pasa bajos sobre las señales originales.

Características utilizadas Esta sección consiste en: El calculo de diversas características en cada sensor y el escalado de las características calculadas.

Selección de características Esta sección consiste en: La implementación de un selector de características, aplicado sobre las características calculadas.

Clasificadores implementados Esta sección consiste en: La implementación de diversos clasificadores sobre el conjunto con las características calculadas y el conjunto de con las características reducidas.

Resultados en validación Esta sección consiste en: La presentación de los resultados obtenidos mediante el uso de los clasificadores en el conjunto de validación.

Resultados en Kaggle Esta sección consiste en: La presentación de los resultados obtenidos en la competencia de Kaggle.

Análisis de los resultados Esta sección consiste en: El análisis de los resultados obtenidos.

Problemas y posibles mejoras Esta sección consiste en: La presentación de los problemas enfrentados durante el desarrollo del proyecto y las posibles mejoras del resultado final.

Índice de Contenidos

1. Desarrollo **1**
1.1. Gráficos de señales . . . 1
1.2. Eliminación de señales . . . 5
1.3. Filtrado de señales . . . 5
1.4. Características utilizadas . . . 10
1.5. Selección de características . . . 12
1.6. Clasificadores implementados . . . 12
1.6.1. Definición del split . . . 12
1.6.2. Modelo Random Forest . . . 12
1.6.3. Modelos SVM . . . 13
1.6.3.1 Modelo SVM Lineal . . . 13
1.6.3.2 Modelo SVM Polinomial . . . 14
1.6.3.3 Modelo SVM Gaussiano . . . 15
1.6.3.4 Modelo SVM Sigmoide . . . 16
1.6.4. Modelo KNN . . . 16
1.6.5. Modelo Naive Bayes . . . 17
1.7. Resultados en validación . . . 18
1.7.1. Métricas . . . 18
1.7.2. Matrices de confusión. . . . 19
1.8. Resultados en Kaggle . . . 21
1.9. Análisis de los resultados . . . 21
1.10. Problemas y posibles mejoras . . . 22

2. Conclusiones **23**
2.1. Aprendizajes . . . 23
2.2. Evaluación de resultados . . . 23

Índice de Figuras

1. Clase 1. . . . 1
2. Clase 4. . . . 2
3. Clase 5. . . . 2
4. Eje x. . . . 3
5. Eje y. . . . 4
6. Eje z. . . . 4
7. Eje x. . . . 7
8. Eje y. . . . 7
9. Eje z. . . . 8
10. Matrices de confusión (1). . . . 19

11. Matrices de confusión (2). 20
12. Matrices de confusión (3). 21

Índice de Tablas

1. Mejores modelos y tiempos de entrenamieno Random Forest. 13
2. Mejores modelos y tiempos de entrenamiento SVM lineal. 14
3. Mejores modelos y tiempos de entrenamiento SVM polinomial. 15
4. Mejores modelos y tiempos de entrenamiento SVM gaussiano. 15
5. Mejores modelos y tiempos de entrenamiento SVM sigmoide. 16
6. Mejores modelos y tiempos de entrenamiento K nearest neighbors. 17
7. Mejores modelos y tiempos de entrenamiento Naive Bayes. 18
8. Métricas. 18
9. Resultados en Kaggle. 21

Índice de Códigos

1. Primera función. 1
2. Segunda función. 2
3. Filtros. 5
4. Función para graficar señales filtradas. 6
5. Función para aplicar el filtro pasa-bajos. 8
6. Filtro aplicado. 9
7. Calculo de características. 10
8. Extracción de características. 11
9. Extracción de características del conjunto de entrenamiento y validación. . . . 11
10. Escalado del conjunto de entrenamiento y validación. 11
11. Selector de características. 12
12. Obtención del split. 12
13. Modelo Random Forest. 12
14. Modelo SVM lineal. 13
15. Modelo SVM polinomial. 14
16. Modelo SVM gaussiano. 15
17. Modelo SVM sigmoide. 16
18. Modelo KNN. 16
19. Modelo naive bayes. 17

1. Desarrollo

1.1. Gráficos de señales

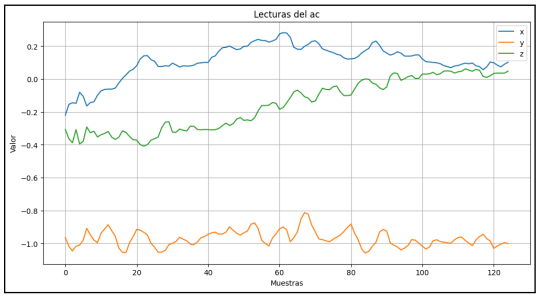
En la presente sección se implementaron dos funciones con el objetivo de visualizar las señales del conjunto de entrenamiento y validación. Para rescatar la información más relevante, las funciones fueron utilizadas sobre las primeras ocurrencias de cada clase.
La primera función implementada grafica todas las señales de un sensor en una misma gráfica, la función implementada se presenta en 1.

Código 1: Primera función.

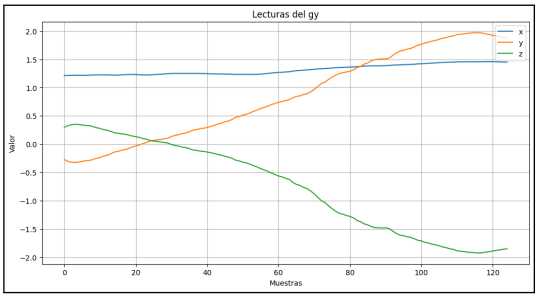
```
def graphs(i, df):
  samples = np.arange(0, 125, 1)

  def maker(IMU):
    X_features = [col for col in df.columns if col.startswith(f'{IMU}x')]
    Y_features = [col for col in df.columns if col.startswith(f'{IMU}y')]
    Z_features = [col for col in df.columns if col.startswith(f'{IMU}z')]
    i_row_IMUx = df[X_features].iloc[i]
    i_row_IMUy = df[Y_features].iloc[i]
    i_row_IMUz = df[Z_features].iloc[i]
```

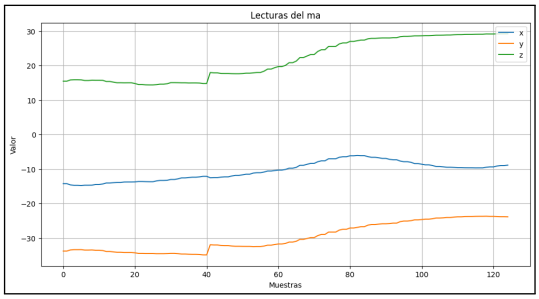
Los resultados sobre las clases 1, 4 y 5 se presentan en 1, 2 y 3, respectivamente.



(a) Acelerómetro.

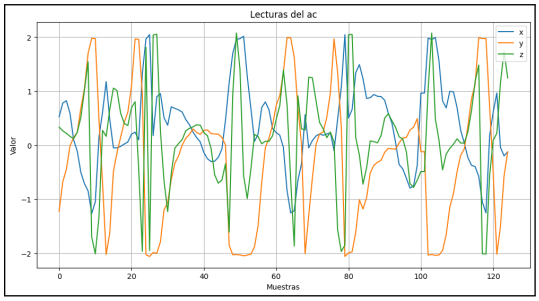


(b) Giróscopio.

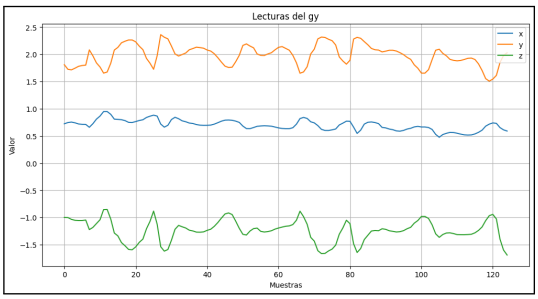


(c) Magnetometro.

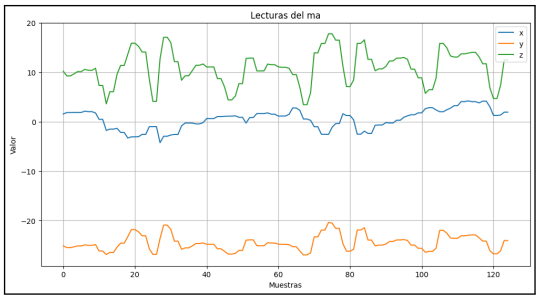
Figura 1: Clase 1.



(a) Acelerómetro.

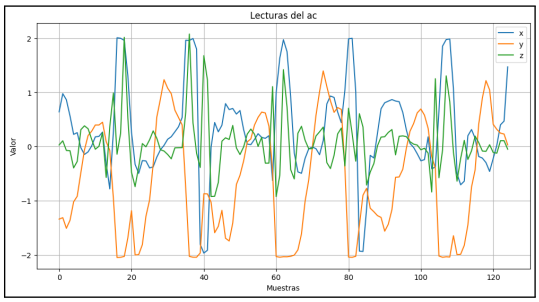


(b) Giróscopio.

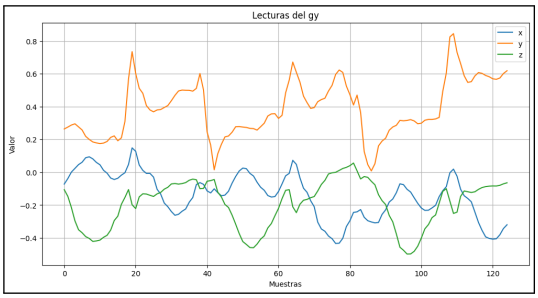


(c) Magnetometro.

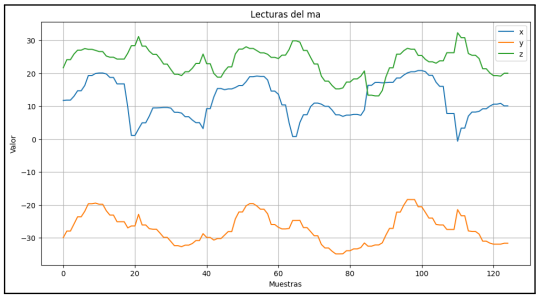
Figura 2: Clase 4.



(a) Acelerómetro.



(b) Giróscopio.



(c) Magnetometro.

Figura 3: Clase 5.

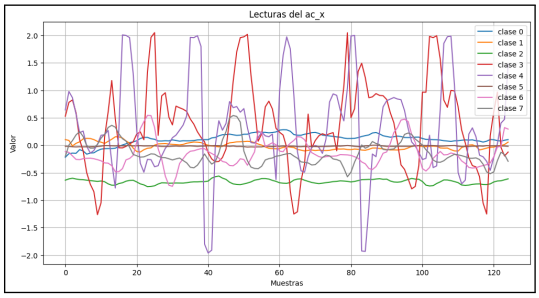
La segunda función implementada grafica todas las señales de un eje para todos los sensores sobre una misma gráfica, la función implementada se presenta en 2.

Código 2: Segunda función.

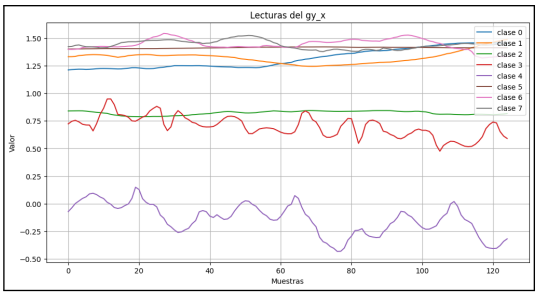
```python
def graphs_2(df,axis):
  samples = np.arange(0, 125, 1)
  ac = []
  gy = []
  ma = []
  ac_features = [col for col in df.columns if col.startswith(f'ac{axis}')]
  gy_features = [col for col in df.columns if col.startswith(f'gy{axis}')]
  ma_features = [col for col in df.columns if col.startswith(f'ma{axis}')]

  for n in range(0, len(df)):
    n_row_ac = df[ac_features].iloc[n].tolist()
    ac.append(n_row_ac)
    n_row_gy = df[gy_features].iloc[n].tolist()
    gy.append(n_row_gy)
    n_row_ma = df[ma_features].iloc[n].tolist()
    ma.append(n_row_ma)
```

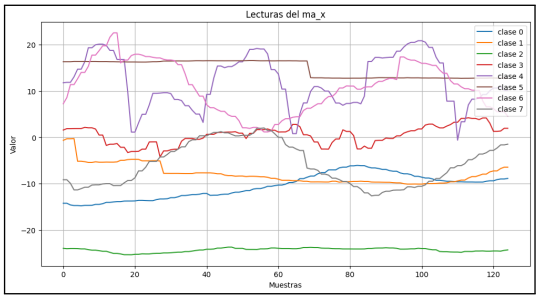
Los resultados del ejes x, y y z se presentan en 4, 5 y 6, respectivamente.



(a) Acelerómetro.

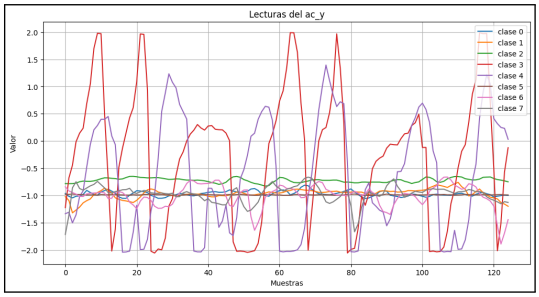


(b) Giróscopio.

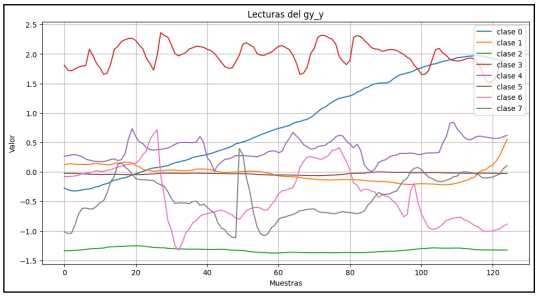


(c) Magnetometro.

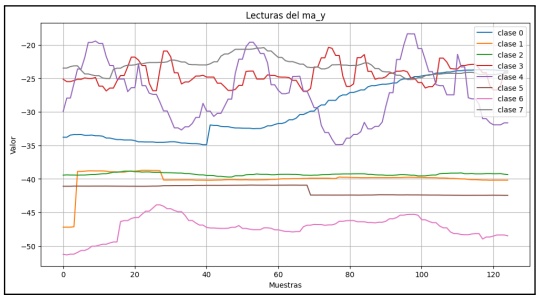
Figura 4: Eje x.



(a) Acelerómetro.

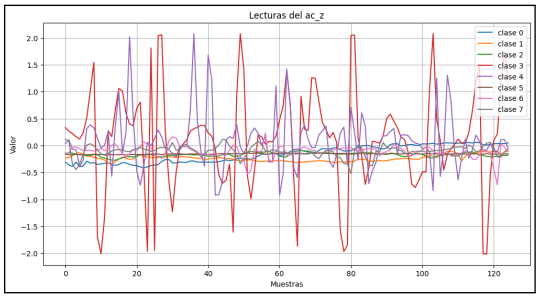


(b) Giróscopio.

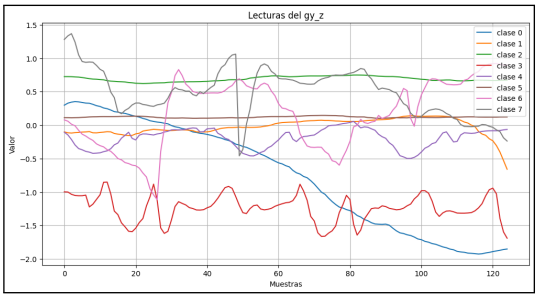


(c) Magnetometro.

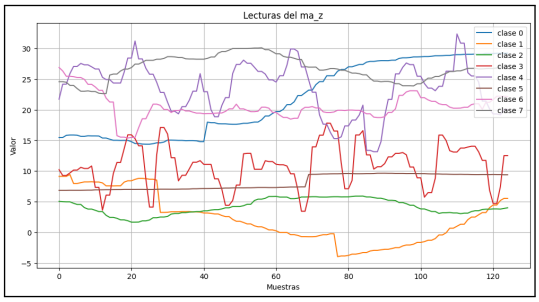
Figura 5: Eje y.



(a) Acelerómetro.



(b) Giróscopio.



(c) Magnetometro.

Figura 6: Eje z.

1.2. Eliminación de señales

En la presente sección se presenta la argumentación de la decisión de no eliminar señales. A partir de 1, 2 y 3 es posible concluir que todas las señales son relevantes para diferenciarlas, y por tanto útiles para la clasificación de las mismas. Si bien existen casos en las señales en el eje y y z de los giroscopios y magnetómetros parecen reflejadas o similares, en otros casos no es así, por lo que eliminar alguna señal podria ser beneficioso para algunas clases y perjudicial para otras.

1.3. Filtrado de señales

En la presente sección se implemento una función con el objetivo de visualizar las señales de cada clase con la aplicación de algún filtro sobre estas. En particular se implementaran filtros pasa-bajos, filtros pasa-altos y filtros pasa-banda.
La implementación de los filtros se presenta en 3.

Código 3: Filtros.

```
cutoff = 2.5
lowcut = 1.0
highcut = 5.0
fs = 60.0

def low_pass_filter(data, cutoff, fs, order=5):
    nyquist = 0.5 * fs
    normal_cutoff = cutoff / nyquist
    b, a = butter(order, normal_cutoff, btype='low', analog=False)
    y = filtfilt(b, a, data)
    return y

def high_pass_filter(data, cutoff, fs, order=5):
    nyquist = 0.5 * fs
    normal_cutoff = cutoff / nyquist
    b, a = butter(order, normal_cutoff, btype='high', analog=False)
    y = filtfilt(b, a, data)
    return y

def band_pass_filter(data, lowcut, highcut, fs, order=5):
    nyquist = 0.5 * fs
    low = lowcut / nyquist
    high = highcut / nyquist
    b, a = butter(order, [low, high], btype='band')
    y = filtfilt(b, a, data)
    return y
```

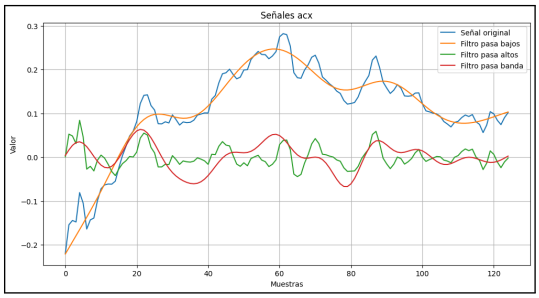
La implementación de la visualización se presenta en 4.

Código 4: Función para graficar señales filtradas.

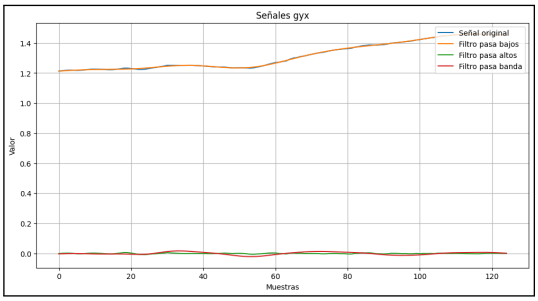
```
def signals(i, df, IMU):
  samples = np.arange(0, 125, 1)
  X_features = [col for col in df.columns if col.startswith(f'{IMU}x')]
  Y_features = [col for col in df.columns if col.startswith(f'{IMU}y')]
  Z_features = [col for col in df.columns if col.startswith(f'{IMU}z')]
  i_row_IMUx = df[X_features].iloc[i]
  i_row_IMUy = df[Y_features].iloc[i]
  i_row_IMUz = df[Z_features].iloc[i]

  low_passx = low_pass_filter(i_row_IMUx, cutoff, fs)
  high_passx = high_pass_filter(i_row_IMUx, cutoff, fs)
  band_passx = band_pass_filter(i_row_IMUx, lowcut, highcut, fs)
  low_passy = low_pass_filter(i_row_IMUy, cutoff, fs)
  high_passy = high_pass_filter(i_row_IMUy, cutoff, fs)
  band_passy = band_pass_filter(i_row_IMUy, lowcut, highcut, fs)
  low_passz = low_pass_filter(i_row_IMUz, cutoff, fs)
  high_passz = high_pass_filter(i_row_IMUz, cutoff, fs)
  band_passz = band_pass_filter(i_row_IMUz, lowcut, highcut, fs)
```

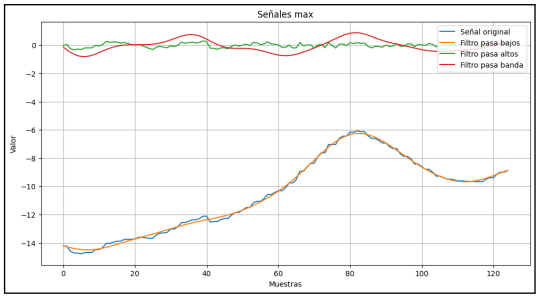
Los resultados del ejes x, y y z de la primera clase se presentan en 7, 8 y 9, respectivamente.



(a) Acelerómetro.

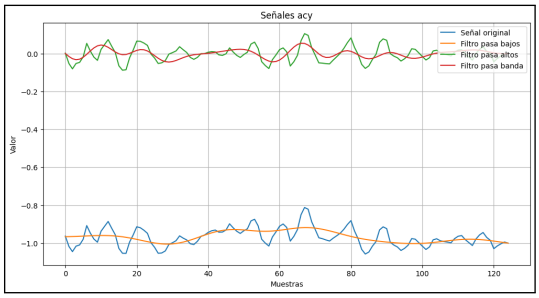


(b) Giróscopio.

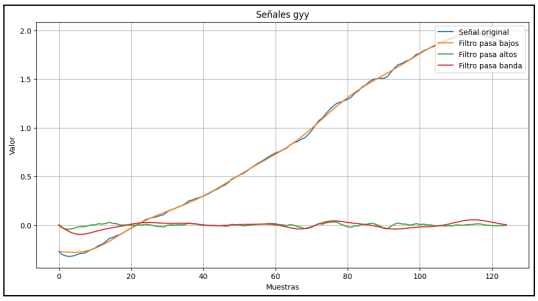


(c) Magnetometro.

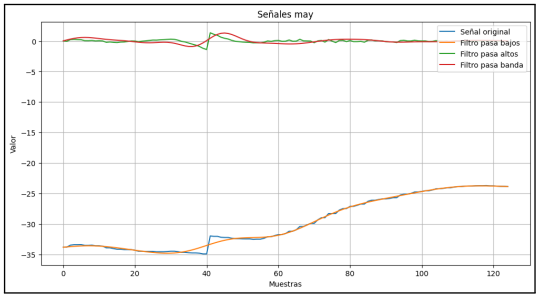
Figura 7: Eje x.



(a) Acelerómetro.

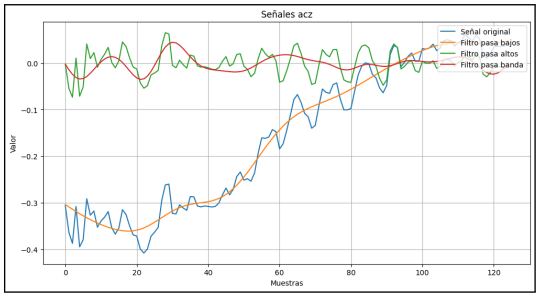


(b) Giróscopio.

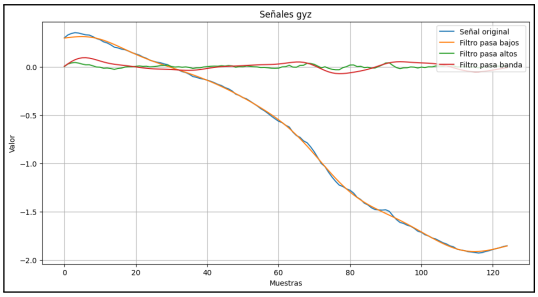


(c) Magnetometro.

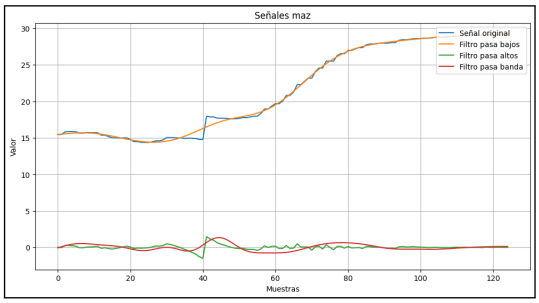
Figura 8: Eje y.



(a) Acelerómetro.



(b) Giróscopio.



(c) Magnetometro.

Figura 9: Eje z.

A partir de los gráficos presentados anteriormente se vislumbra que la mejor opción es utilizar el filtro pasa-bajos, pues mantiene la forma de la señal y elimina el ruido posiblemente perjudicial.
Para aplicar el filtro pasa-bajos sobre los dataframes de entrenamiento y validación se implemento 5.

Código 5: Función para aplicar el filtro pasa-bajos.

```
def apply_low_pass_filter(df, cutoff, fs):
    filtered_df = df.copy()
    X_ac_features = [col for col in df.columns if col.startswith('acx')]
    Y_ac_features = [col for col in df.columns if col.startswith('acy')]
    Z_ac_features = [col for col in df.columns if col.startswith('acz')]
    X_gy_features = [col for col in df.columns if col.startswith('gyx')]
    Y_gy_features = [col for col in df.columns if col.startswith('gyy')]
    Z_gy_features = [col for col in df.columns if col.startswith('gyz')]
    X_ma_features = [col for col in df.columns if col.startswith('max')]
    Y_ma_features = [col for col in df.columns if col.startswith('may')]
    Z_ma_features = [col for col in df.columns if col.startswith('maz')]

    for i in range(len(df)):
      row_acx = filtered_df[X_ac_features].iloc[i].values
      row_acy = filtered_df[Y_ac_features].iloc[i].values
      row_acz = filtered_df[Z_ac_features].iloc[i].values
      row_gyx = filtered_df[X_gy_features].iloc[i].values
      row_gyy = filtered_df[Y_gy_features].iloc[i].values
```

```
    row_gyz = filtered_df[Z_gy_features].iloc[i].values
    row_max = filtered_df[X_ma_features].iloc[i].values
    row_may = filtered_df[Y_ma_features].iloc[i].values
    row_maz = filtered_df[Z_ma_features].iloc[i].values

    row = []
    low_pass_ac_x = low_pass_filter(row_acx, cutoff, fs)
    low_pass_ac_y = low_pass_filter(row_acy, cutoff, fs)
    low_pass_ac_z = low_pass_filter(row_acz, cutoff, fs)
    low_pass_gy_x = low_pass_filter(row_gyx, cutoff, fs)
    low_pass_gy_y = low_pass_filter(row_gyy, cutoff, fs)
    low_pass_gy_z = low_pass_filter(row_gyz, cutoff, fs)
    low_pass_ma_x = low_pass_filter(row_max, cutoff, fs)
    low_pass_ma_y = low_pass_filter(row_may, cutoff, fs)
    low_pass_ma_z = low_pass_filter(row_maz, cutoff, fs)

    row.extend(low_pass_ac_x)
    row.extend(low_pass_ac_y)
    row.extend(low_pass_ac_z)
    row.extend(low_pass_gy_x)
    row.extend(low_pass_gy_y)
    row.extend(low_pass_gy_z)
    row.extend(low_pass_ma_x)
    row.extend(low_pass_ma_y)
    row.extend(low_pass_ma_z)

    filtered_df.iloc[i, 1:1227] = row

  return filtered_df
```

Finalmente la aplicación de los filtros se presenta en 6.

Código 6: Filtro aplicado.

```
filtered_X_train = apply_low_pass_filter(X_train, cutoff, fs)
filtered_X_val = apply_low_pass_filter(X_val, cutoff, fs)
```

1.4. Características utilizadas

En la presente sección se implemento la extracción de características y el escalado de los conjuntos de datos
Las características utilizadas se presentan a continuación.

Media Promedio aritmético de los valores en una señal.

Varianza Mide la dispersión de los valores en una señal respecto a la media.

Mínimo El valor más pequeño en una señal.

Máximo El valor más grande en una señal.

Raíz cuadrática media Raíz cuadrada de la media de los cuadrados de los valores de la señal.

Asimetría Simetría de la distribución de los valores de la señal.

Curtosis Mide la altura y el ancho de la señal, indicando cuan concentrados están dichos valores en torno a la media.

Correlación Auto correlación de la señal.

Entropía promedio Mide la incertidumbre o aleatoriedad en la distribución de los valores de la señal.

La implementación de las funciones para el calculo y la extracción de las características se presentan en 7 y 8, respectivamente.

Código 7: Calculo de características.

```
def calculate_features(sequence):
    features = {}
    features['mean'] = np.mean(sequence)
    features['variance'] = np.var(sequence)
    features['min'] = np.min(sequence)
    features['max'] = np.max(sequence)
    features['rms'] = np.sqrt(np.mean(sequence**2))
    features['skew'] = skew(sequence)
    features['kurtosis'] = kurtosis(sequence)
    features['correlation'] = np.correlate(sequence, sequence, mode='full')[len(sequence)-1] /
     ↪ len(sequence)
    hist, _ = np.histogram(sequence, bins=10, density=True)
    hist = hist[hist > 0]
    features['average_entropy'] =  -np.sum(hist * np.log(hist))
    return features
```

Código 8: Extracción de características.

```
def extract_features_for_sensors(df):
    feature_dict = {}
    sensors = ['acx', 'acy', 'acz', 'gyx', 'gyy', 'gyz', 'max', 'may', 'maz']
    for sensor in sensors:
        columns = [f"{sensor}{i}" for i in range(125)]
        sequence = df[columns].values
        features = calculate_features(sequence)
        feature_dict.update({f"{sensor}_{k}": v for k, v in features.items()})
    return pd.Series(feature_dict)
```

La extracción de las características del conjunto de entrenamiento y validación se presenta en 9.

Código 9: Extracción de características del conjunto de entrenamiento y validación.

```
features_train = filtered_X_train.apply(extract_features_for_sensors, axis=1)
features_val = filtered_X_val.apply(extract_features_for_sensors, axis=1)
```

Finalmente se realiza el escalado de los conjuntos de datos mediante *standard scaler* se presenta en 10.

Código 10: Escalado del conjunto de entrenamiento y validación.

```
Scaler = StandardScaler()
Scaler.fit(features_train)
features_train_normalized = Scaler.transform(features_train)
features_val_normalized = Scaler.transform(features_val)
df_train_normalized = pd.DataFrame(features_train_normalized, columns=features_train.
    ↪ columns)
df_val_normalized = pd.DataFrame(features_val_normalized, columns=features_val.
    ↪ columns)
df_train_normalized.index = filtered_X_train.index
df_val_normalized.index = filtered_X_val.index
```

1.5. Selección de características

En la presente sección se implemento un selector de características. En particular se implemento un selector tipo *SelectKBest* con k igual a la mitad de las características calculadas previamente. La implementación del selector se presenta en 11.

Código 11: Selector de características.

```
selector = SelectKBest(score_func=f_classif, k=int(len(df_train_normalized.columns) *
    ↪ 0.5))
reduced_train = selector.fit_transform(df_train_normalized, y_train)
reduced_val = selector.fit_transform(df_val_normalized, y_val)
```

1.6. Clasificadores implementados

En la presente sección se explica la metodología implementada para el entrenamiento de los clasificadores con excepción del clasificador Naive Bayes.
Para todos los clasificadores se utilizo *GridSearchCV* para obtener los mejores hiper parámetros. Luego se entreno el mejor clasificador con el conjunto de entrenamiento. Finalmente se realiza la predicción sobre el conjunto de validación.

1.6.1. Definición del split

El split utilizado en el *GridSearchCV* es el mismo que fue implementado en las tareas a lo largo del curso.
La implementación se presenta en 12.

Código 12: Obtención del split.

```
indexs = np.zeros(len(df_train_normalized) + len(df_val_normalized))
indexs[:len(df_train_normalized)] = -1
cv = PredefinedSplit(test_fold=indexs)
```

1.6.2. Modelo Random Forest

El modelo Random Forest implementado es *RandomForestClassifier* de *scikit-learn*. Los hiper parámetros a definir mediante la búsqueda de la grilla son *max_depth* y *n_estimators*. La implementación del modelo, su entrenamiento y la búsqueda de la grilla se presentan en 13.

Código 13: Modelo Random Forest.

```
rf_model = RandomForestClassifier(random_state=3)
parameters_grid_rf = {'max_depth':[10, 15, 20, 25, 30],'n_estimators': [50, 100, 150, 200,
    ↪ 250]}
grid_search_rf = GridSearchCV(rf_model, param_grid=parameters_grid_rf, cv=cv)
start = time.time()
```

```
grid_search_rf.fit(np.vstack((df_train_normalized, df_val_normalized)), np.hstack((y_train , y_val)))
end = time.time()
best_rf = grid_search_rf.best_estimator_
best_rf.fit(df_train_normalized, y_train)
y_pred = best_rf.predict(df_val_normalized)
grid_search_rf_k_best = GridSearchCV(rf_model, param_grid=parameters_grid_rf, cv= cv)
start = time.time()
grid_search_rf_k_best.fit(np.vstack((reduced_train, reduced_val)), np.hstack((y_train, y_val)))
end = time.time()
best_rf_k_best = grid_search_rf_k_best.best_estimator_
best_rf_k_best.fit(reduced_train, y_train)
y_pred = best_rf_k_best.predict(reduced_val)
```

Los mejores modelos y sus tiempos de entrenamiento se presentan en 1.

Tabla 1: Mejores modelos y tiempos de entrenamieno Random Forest.

Características	Mejores hiper parámetros	Tiempo de entrenamiento
Totales	max_depth = 15, n_estimators = 200	89.78054 segundos
Reducidas	max_depth = 10, n_estimators = 100	59.2830 segundos

1.6.3. Modelos SVM

1.6.3.1. Modelo SVM Lineal

El modelo SVM con kernel lineal implementado es un *SVC* de *scikit-learn*. El hiper parámetro a definir mediante la búsqueda de la grilla es *C*.
La implementación del modelo, su entrenamiento y la búsqueda de la grilla se presentan en 14.

Código 14: Modelo SVM lineal.

```
svm_linear_model = SVC(kernel= 'linear', random_state=3)
parameters_grid_svm = {'C': [0.01, 0.1, 1, 10, 100]}
grid_search_svm = GridSearchCV(svm_linear_model, param_grid=parameters_grid_svm, cv=cv)
start = time.time()
grid_search_svm.fit(np.vstack((df_train_normalized, df_val_normalized)), np.hstack(( y_train, y_val)))
end = time.time()
best_linear_svm = grid_search_svm.best_estimator_
best_linear_svm.fit(df_train_normalized, y_train)
y_pred = best_linear_svm.predict(df_val_normalized)
grid_search_svm_k_best = GridSearchCV(svm_linear_model, param_grid=
```

```
    ↪ parameters_grid_svm, cv=cv)
start = time.time()
grid_search_svm_k_best.fit(np.vstack((reduced_train, reduced_val)), np.hstack((y_train,
    ↪ y_val)))
end = time.time()
best_linear_svm_k_best = grid_search_svm_k_best.best_estimator_
best_linear_svm_k_best.fit(reduced_train, y_train)
y_pred = best_linear_svm_k_best.predict(reduced_val)
```

Los mejores modelos y sus tiempos de entrenamiento se presentan en 2.

Tabla 2: Mejores modelos y tiempos de entrenamiento SVM lineal.

Características	Mejores hiper parámetros	Tiempo de entrenamiento
Totales	C = 1	0.7912 segundos
Reducidas	C = 0.01	0.6228 segundos

1.6.3.2. Modelo SVM Polinomial

El modelo SVM con kernel polinomial implementado es un *SVC* de *scikit-learn*. Los hiper parámetros a definir mediante la búsqueda de la grilla son *C*, *degree* y *coef0*.
La implementación del modelo, su entrenamiento y la búsqueda de la grilla se presentan en 15.

Código 15: Modelo SVM polinomial.

```
svm_poly_model = SVC(kernel='poly', random_state=3)
parameters_grid_svm = {'C': [0.01, 0.1, 1, 10], 'degree': [2, 3, 4, 5], 'coef0': [1, 5, 10, 20]}
grid_search_svm = GridSearchCV(svm_poly_model, param_grid=parameters_grid_svm,
    ↪ cv=cv)
start = time.time()
grid_search_svm.fit(np.vstack((df_train_normalized, df_val_normalized)), np.hstack((
    ↪ y_train, y_val)))
end = time.time()
best_poly_svm = grid_search_svm.best_estimator_
best_poly_svm.fit(df_train_normalized, y_train)
y_pred = best_poly_svm.predict(df_val_normalized)
grid_search_poly_svm_k_best = GridSearchCV(svm_poly_model, param_grid=
    ↪ parameters_grid_svm, cv=cv)
start = time.time()
grid_search_poly_svm_k_best.fit(np.vstack((reduced_train, reduced_val)), np.hstack((
    ↪ y_train, y_val)))
end = time.time()
best_poly_svm_k_best.fit(reduced_train, y_train)
y_pred = best_poly_svm_k_best.predict(reduced_val)
```

Los mejores modelos y sus tiempos de entrenamiento se presentan en 3.

Tabla 3: Mejores modelos y tiempos de entrenamiento SVM polinomial.

Características	Mejores hiper parámetros	Tiempo de entrenamiento
Totales	$C = 0.01, coef0 = 5, degree = 2$	15.5526 segundos
Reducidas	$C = 0.01, coef0 = 20, degree = 2$	11.7927 segundos

1.6.3.3. Modelo SVM Gaussiano

El modelo SVM con kernel gaussiano implementado es un *SVC* de *scikit-learn*. Los hiper parámetros a definir mediante la búsqueda de la grilla son C y *gamma*.
La implementación del modelo, su entrenamiento y la búsqueda de la grilla se presentan en 16.

Código 16: Modelo SVM gaussiano.

```
svm_rbf_model = SVC(kernel='rbf', random_state=3)
parameters_grid_svm = {'C': [0.1, 1, 10, 100], 'gamma': [0.001, 0.01, 0.1]}
grid_search_svm = GridSearchCV(svm_rbf_model, param_grid=parameters_grid_svm, cv
    ↪ =cv)
start = time.time()
grid_search_svm.fit(np.vstack((df_train_normalized, df_val_normalized)), np.hstack((
    ↪ y_train, y_val)))
end = time.time()
best_rbf_svm = grid_search_svm.best_estimator_
best_rbf_svm.fit(df_train_normalized, y_train)
y_pred = best_rbf_svm.predict(df_val_normalized)
grid_search_svm_k_best = GridSearchCV(svm_rbf_model, param_grid=
    ↪ parameters_grid_svm, cv=cv)
start = time.time()
grid_search_svm_k_best.fit(np.vstack((reduced_train, reduced_val)), np.hstack((y_train,
    ↪ y_val)))
end = time.time()
best_rbf_svm_k_best = grid_search_svm_k_best.best_estimator_
best_rbf_svm_k_best.fit(reduced_train, y_train)
y_pred = best_rbf_svm_k_best.predict(reduced_val)
```

Los mejores modelos y sus tiempos de entrenamiento se presentan en 4.

Tabla 4: Mejores modelos y tiempos de entrenamiento SVM gaussiano.

Características	Mejores hiper parámetros	Tiempo de entrenamiento
Totales	$C = 1, gamma = 0.001$	13.9317 segundos
Reducidas	$C = 10, gamma = 0.001$	4.8679 segundos

1.6.3.4. Modelo SVM Sigmoide

El modelo SVM con kernel sigmoide implementado es un *SVC* de *scikit-learn*. Los hiper parámetros a definir mediante la búsqueda de la grilla son *C*, *gamma* y *coef0*.
La implementación del modelo, su entrenamiento y la búsqueda de la grilla se presentan en 17.

Código 17: Modelo SVM sigmoide.

```
svm_sigmoid_model = SVC(kernel='sigmoid', random_state=3)
parameters_grid_svm = {'C': [0.1, 1, 10, 100], 'gamma': [0.001, 0.01, 0.1, 1], 'coef0': [0.01,
    ↪ 0.1, 1, 10]}
grid_search_svm = GridSearchCV(svm_sigmoid_model, param_grid=
    ↪ parameters_grid_svm, cv=cv)
start = time.time()
grid_search_svm.fit(np.vstack((df_train_normalized, df_val_normalized)), np.hstack((
    ↪ y_train, y_val)))
end = time.time()
best_sigmoid_svm = grid_search_svm.best_estimator_
best_sigmoid_svm.fit(df_train_normalized, y_train)
y_pred = best_sigmoid_svm.predict(df_val_normalized)
grid_search_svm_k_best = GridSearchCV(svm_sigmoid_model, param_grid=
    ↪ parameters_grid_svm, cv=cv)
start = time.time()
grid_search_svm_k_best.fit(np.vstack((reduced_train, reduced_val)), np.hstack((y_train,
    ↪ y_val)))
end = time.time()
best_sigmoid_svm_k_best = grid_search_svm_k_best.best_estimator_
best_sigmoid_svm_k_best.fit(reduced_train, y_train)
y_pred = best_sigmoid_svm_k_best.predict(reduced_val)
```

Los mejores modelos y sus tiempos de entrenamiento se presentan en 5.

Tabla 5: Mejores modelos y tiempos de entrenamiento SVM sigmoide.

Características	Mejores hiper parámetros	Tiempo de entrenamiento
Totales	$C = 1, coef0 = 0.01, gamma = 0.001$	67.6075 segundos
Reducidas	$C = 100, coef0 = 0.1, gamma = 0.01$	75.3650 segundos

1.6.4. Modelo KNN

El modelo KNN implementado es un *KNeighborsClassifier* de *scikit-learn*. Los hiper parámetros a definir mediante la búsqueda de la grilla son *n_neighbors*, *weights* y *metric*.
La implementación del modelo, su entrenamiento y la búsqueda de la grilla se presentan en 18.

Código 18: Modelo KNN.

```
knn_model = KNeighborsClassifier()
parameters_grid_knn = {'n_neighbors': [6, 9, 12, 15, 18], 'weights': ['uniform', 'distance'],'
    ↪ metric': ['euclidean', 'manhattan', 'minkowski']}
grid_search_knn = GridSearchCV(knn_model, param_grid=parameters_grid_knn, cv=cv)
start = time.time()
grid_search_knn.fit(np.vstack((df_train_normalized, df_val_normalized)), np.hstack((
    ↪ y_train, y_val)))
end = time.time()
best_knn = grid_search_knn.best_estimator_
best_knn.fit(df_train_normalized, y_train)
y_pred = best_knn.predict(df_val_normalized)
grid_search_knn_k_best = GridSearchCV(knn_model, param_grid=parameters_grid_knn
    ↪ , cv=cv)
start = time.time()
grid_search_knn_k_best.fit(np.vstack((reduced_train, reduced_val)), np.hstack((y_train,
    ↪ y_val)))
end = time.time()
best_knn_k_best = grid_search_knn_k_best.best_estimator_
best_knn_k_best.fit(reduced_train, y_train)
y_pred = best_knn_k_best.predict(reduced_val)
```

Los mejores modelos y sus tiempos de entrenamiento se presentan en 6.

Tabla 6: Mejores modelos y tiempos de entrenamiento K nearest neighbors.

Características	Mejores hiper parámetros	Tiempo de entrenamiento
Totales	metric = Manhattan, n_neighbors = 18, weights = uniform	8.9115 segundos
Reducidas	metric = Manhattan, n_neighbors = 6, weights = distance	3.9135 segundos

1.6.5. Modelo Naive Bayes

El modelo naive bayes implementado es un *GaussianNB* de *scikit-learn*.
La implementación del modelo y su entrenamiento se presentan en 19.

Código 19: Modelo naive bayes.

```
gnb = GaussianNB()
start = time.time()
gnb.fit(df_train_normalized, y_train)
end = time.time()
y_pred = gnb.predict(df_val_normalized)
start = time.time()
gnb.fit(reduced_train, y_train)
end = time.time()
y_pred = gnb.predict(reduced_val)
```

Los modelos y sus tiempos de entrenamiento se presentan en 7.

Tabla 7: Mejores modelos y tiempos de entrenamiento Naive Bayes.

Características	Tiempo de entrenamiento
Totales	0.8775 segundos
Reducidas	0.6972 segundos

1.7. Resultados en validación

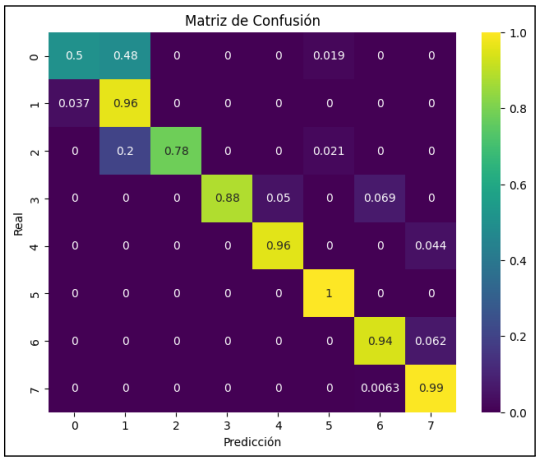
En la presente sección se presentan los resultados de los modelos en el conjunto de validación.
Las métricas obtenidas en cada modelo se presentan en 1.7.1.
Las matrices de confusión obtenidas en cada modelo se presentan en 1.7.2.

1.7.1. Métricas

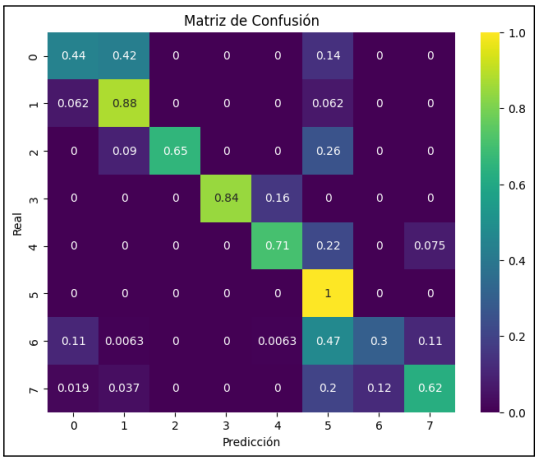
Tabla 8: Métricas.

Modelo	Accuracy	Precision	Recall
RF	0.88	0.91	0.88
RF K Best	0.68	0.76	0.68
SVM Lineal	0.84	0.88	0.84
SVM Lineal K Best	0.75	0.79	0.75
SVM Polinomial	0.92	0.94	0.92
SVM Polinomial K Best	0.75	0.79	0.75
SVM Gaussiano	0.92	0.94	0.92
SVM Gaussiano K Best	0.75	0.78	0.75
SVM Sigmoide	0.91	0.94	0.91
SVM Sigmoide K Best	0.79	0.84	0.79
KNN	0.83	0.85	0.83
KNN K Best	0.73	0.78	0.73
NB	0.85	0.87	0.85
NB K Best	0.16	0.07	0.16

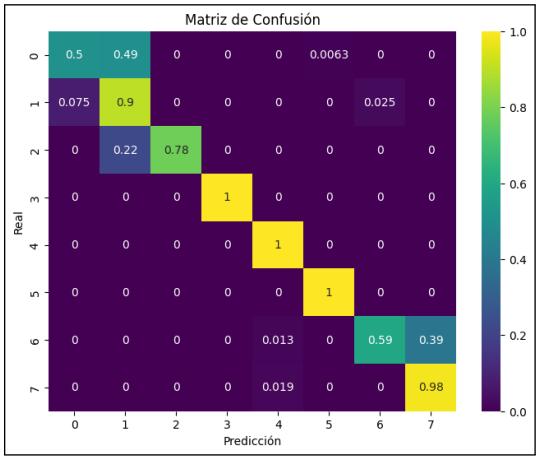
1.7.2. Matrices de confusión.



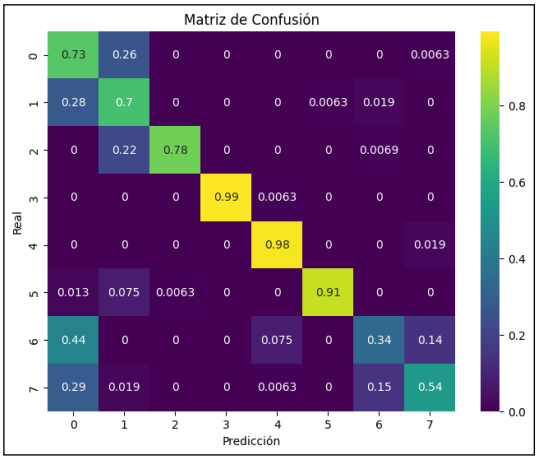
(a) RF base.



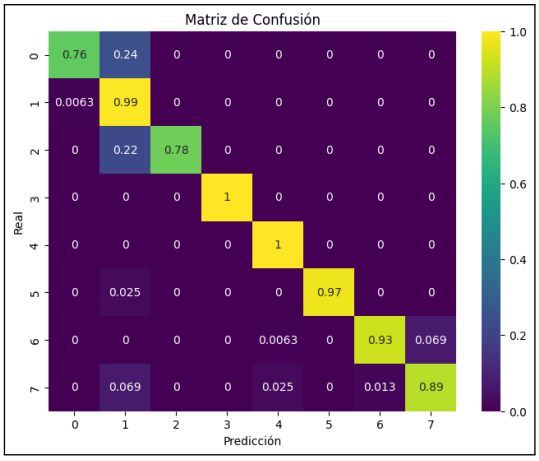
(b) RF reducido.



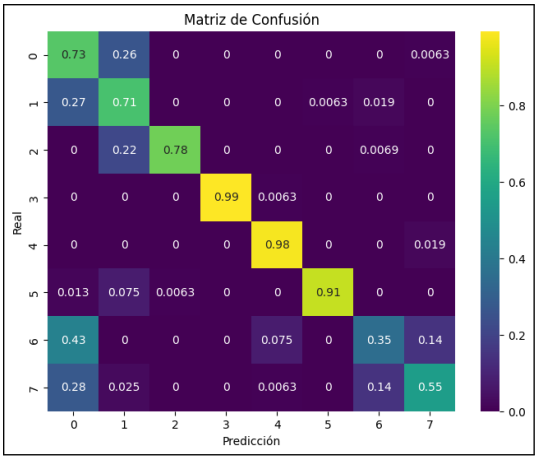
(c) SVM lineal base.



(d) SVM lineal reducido.

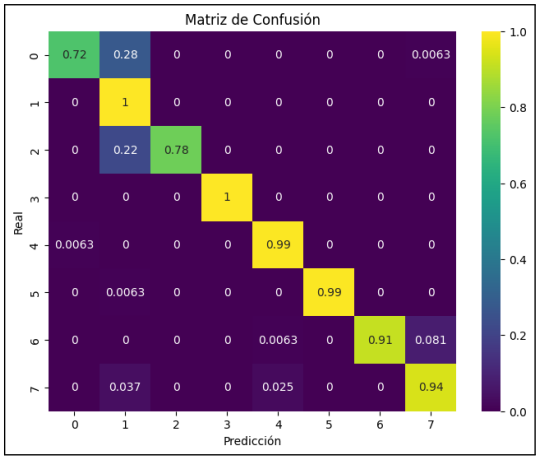


(e) SVM polinomial base.

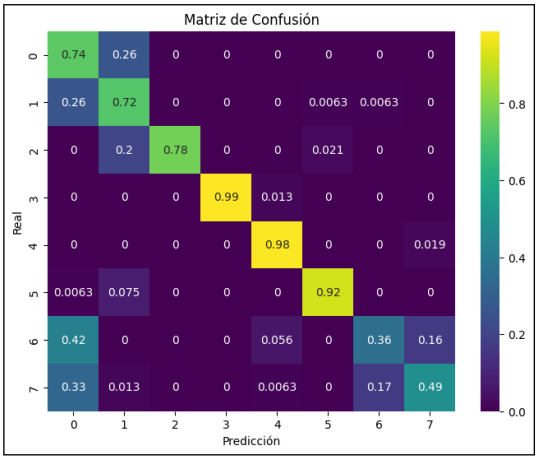


(f) SVM polinomial reducido.

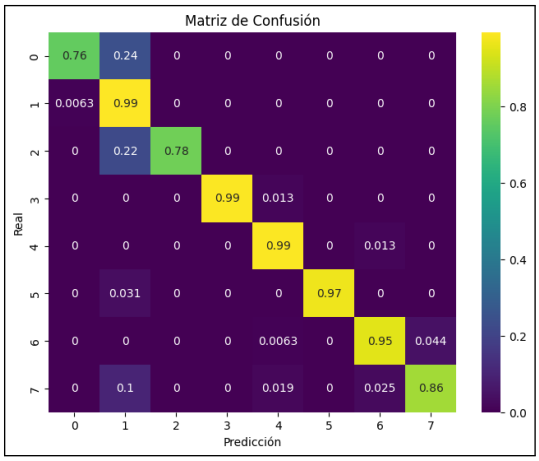
Figura 10: Matrices de confusión (1).



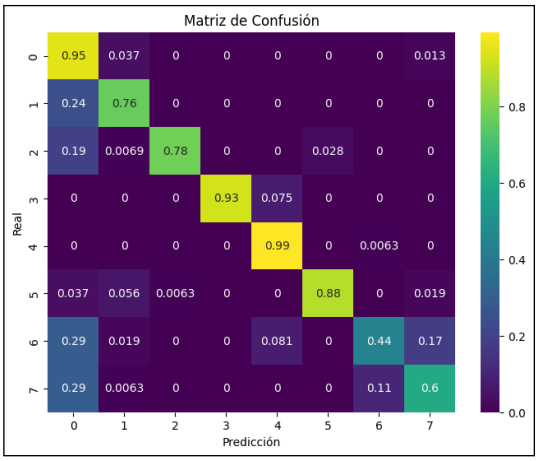
(a) SVM gaussiano base.



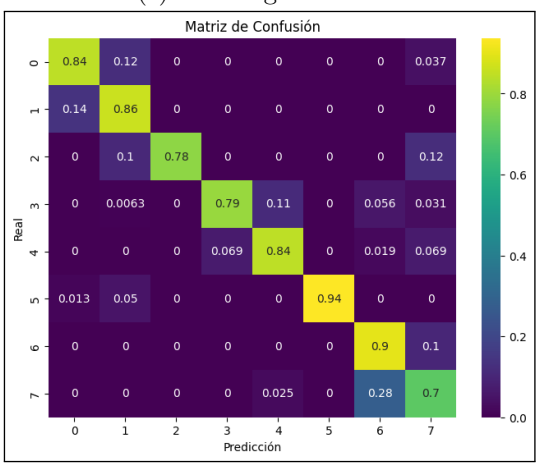
(b) SVM gaussiano reducido.



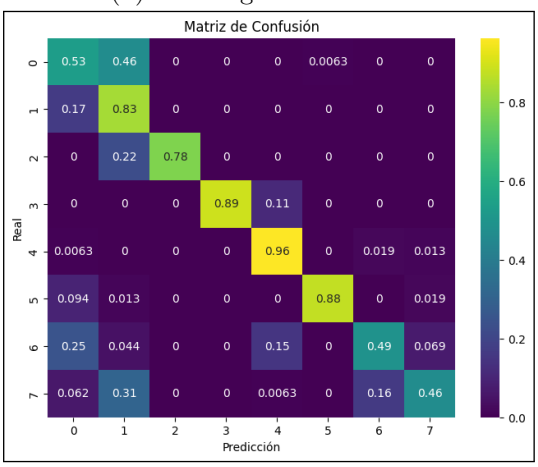
(c) SVM sigmoide base.



(d) SVM sigmoide reducido.

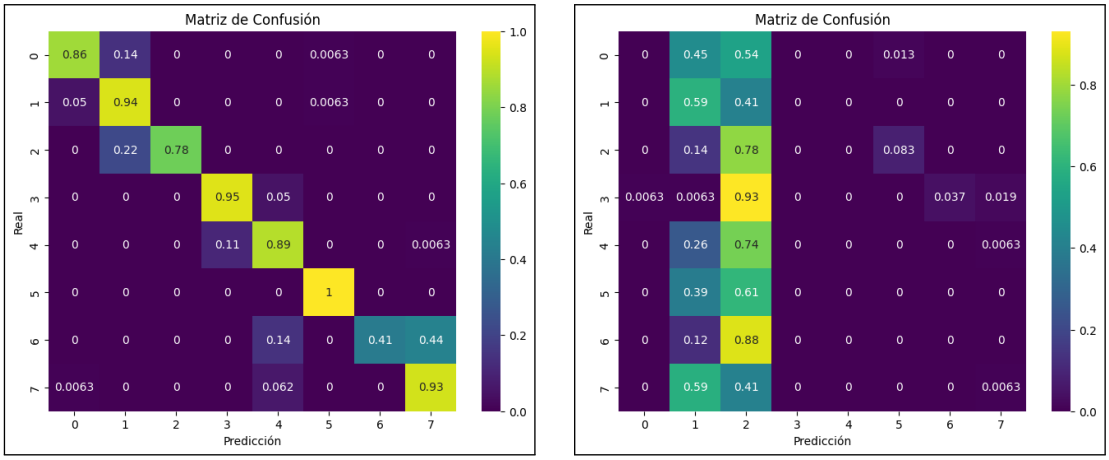


(e) KNN base.



(f) KNN reducido.

Figura 11: Matrices de confusión (2).



(a) NB base.

(b) NB reducido.

Figura 12: Matrices de confusión (3).

1.8. Resultados en Kaggle

En la presente sección se presentan los resultados obtenidos al aplicar el mejor clasificador obtenido sobre el conjunto de entrenamiento.
El mejor clasificador y por tanto el aplicado sobre el conjunto de entrenamiento es el Support Vector Machine con kernel rbf presentado en la sección 1.6.3.3. Se utilizo el total de las características calculadas en 1.4.
Los resultados obtenidos al momento de realizar el informe se presentan en 9.

Tabla 9: Resultados en Kaggle.

Ranking	Score
20	0.78229

1.9. Análisis de los resultados

En la presente sección se presenta el análisis de los resultados obtenidos tanto en el conjunto de validación y como en el de prueba.
Para el caso del conjunto de validación, se analizaran los 3 mejores clasificadores. En este problema los clasificadores que obtuvieron las mejores métricas son los SVM con kernel: Gaussiano, Polinomial y Sigmoide. Una posible explicación de estos resultados es la capacidad de los kernels utilizados para capturar relaciones no lineales entre los datos, sumado a la eficiencia de los Support Vector Machines para manejar los espacios de alta dimensionalidad (en este caso 81 características). Además, en comparación con el resto de los modelos implementados se tiene que: en el caso del modelo Random Forest, este posee una menor capacidad para capturar las relaciones no lineales de los datos, en el caso del modelo KNN, este posee un desempeño disminuido al momento de utilizarse sobre espacios de alta dimensionalidad y en el caso del modelo Naive Bayes, este asume independencia entre las características, el

cual no es el caso del problema.
Para el caso del conjunto de prueba, se analizaran los resultados obtenidos sobre este. Al momento de utilizar el mejor clasificador sobre el conjunto de prueba, se obtuvo un accuracy del 0.78229 sobre un 75 % del conjunto. El resultado obtenido no es un valor bajo pero dista bastante del resultado obtenido por el resto de mis compañeros. La explicación más probable de este bajo desempeño es que en los clasificadores entrenados se produjo overfitting. Las posibles causales de esto son la cantidad de características y la complejidad de estas mismas.

1.10. Problemas y posibles mejoras

Los problemas enfrentados al momento de realizar el proyecto se describen a continuación.

RNN Esta fue la primera aproximación que intente para resolver el problema, obteniendo resultados similares a los obtenidos con el clasificador SVM gaussiano, pero empleando más tiempo en la implementación del RNN. El principal problema fue el desconocimiento de como se implementan correctamente dichas redes, pues la implementación que realice tomaba como secuencias cada eje del sensor y no la secuencia temporal de todos los sensores en conjunto. La errada implementación resulto en overfitting, similar al caso del SVM gaussiano. Al momento de intentar implementar la RNN de forma correcta quedaba poco tiempo para la entrega del proyecto, por lo que decidí implementar los clasificadores presentados en este informe.

Aplicación de un filtro Previo a realizar el proyecto desconocía como realizar un filtrado de datos. Realizar esto me tomo bastante tiempo, el cual hubiese sido destinado a la implementación de los clasificadores.

Definición de las características El desconocimiento de la extracción de características resulto en la implementación de las características que parecían ser las mejores para las señales obtenidas mediante el filtro, pero es probable que la cantidad utilizada y la complejidad de algunas de estas hayan resultado en el overfitting de los clasificadores.

Conjunto de prueba Al momento de realizar el calculo de las clases del conjunto de pruebas cometi varias veces el error de obtener las clases restadas en uno, esto por el hecho de inicialmente utilizar un RNN, pues para dicho caso las clases deben comenzar desde el cero, por lo que luego debía sumarle uno. Además, al momento de cambiar los clasificadores olvide eliminar el restado, lo que ocasiono que nuevamente obtuviera resultados errados.

Poca búsqueda de información Al momento de afrontar el problema a resolver no analice con profundidad el problema para posteriormente desarrollar clasificadores, si no que hice lo contrario, comencé la implementación sin el análisis en profundidad del problema, lo cual resulto en resultados incorrectos. Mientras enfrentaba el problema me documentaba, lo cual no fue la aproximación correcta.

Las posibles mejoras se listan a continuación.

- Mayor análisis del problema.
- Mayor documentación previo al desarrollo.
- Probar más características o distintas selecciones de estas.
- Realizar más intentos en Kaggle.
- Probar distintos tamaños para el conjunto de validación.
- Implementar otros clasificadores.

2. Conclusiones

2.1. Aprendizajes

A continuación, se describen mis aprendizajes.

Análisis y documentación del problema He aprendido acerca de la importancia de realizar un análisis en profundidad del problema a resolver previo al desarrollo del mismo. He aprendido que en ocasiones un problema simple no requiere una solución muy elaborada. He aprendido que es bueno realizar una amplia documentación acerca de problemas similares fructífero para el desarrollo del mismo.

Redes Deep Learning He aprendido acerca de las redes RNN y la utilidad de estas en problemas temporales. He aprendido a que en ocasiones la complejidad de estas puede producir overfitting en problemas relativamente sencillos.

Extracción de características He aprendido acerca de la implementación y utilidad de la extracción de características. En particular, la utilidad para reducir o aumentar la dimensionalidad de un problema.

Kaggle He aprendido acerca de la existencia de kaggle y la relevancia de esta plataforma web en los ámbitos de la inteligencia computacional, data science y el deep learning.

2.2. Evaluación de resultados

Los resultados obtenidos son ambiguos según a los esperados en función de la documentación accedida. Según esta misma, los modelos que debiesen obtener los mejores resultados son los de deep learning, el cual no fue el caso al momento de implementar dichos modelos. Respecto al caso de extracción de características, los resultados si son los esperados, pues según la documentación los mejores clasificadores son los SVM, seguido por los Random Forest y luego los KNN, por tanto en este casos si son los resultados esperados.
Finalmente, respecto a los resultados obtenidos en la implementación de una red deep learning y un clasificador tradicional con extracción de características se aprecia que los resultados obtenidos son iguales, por tanto se comprueba que la extracción de características es

una herramienta útil para reducir la dimensionalidad del problema sin perder la información del conjunto de datos en el proceso.